CS 214: Data Structures

Recursion & Stacks Applications

Slide contents follow

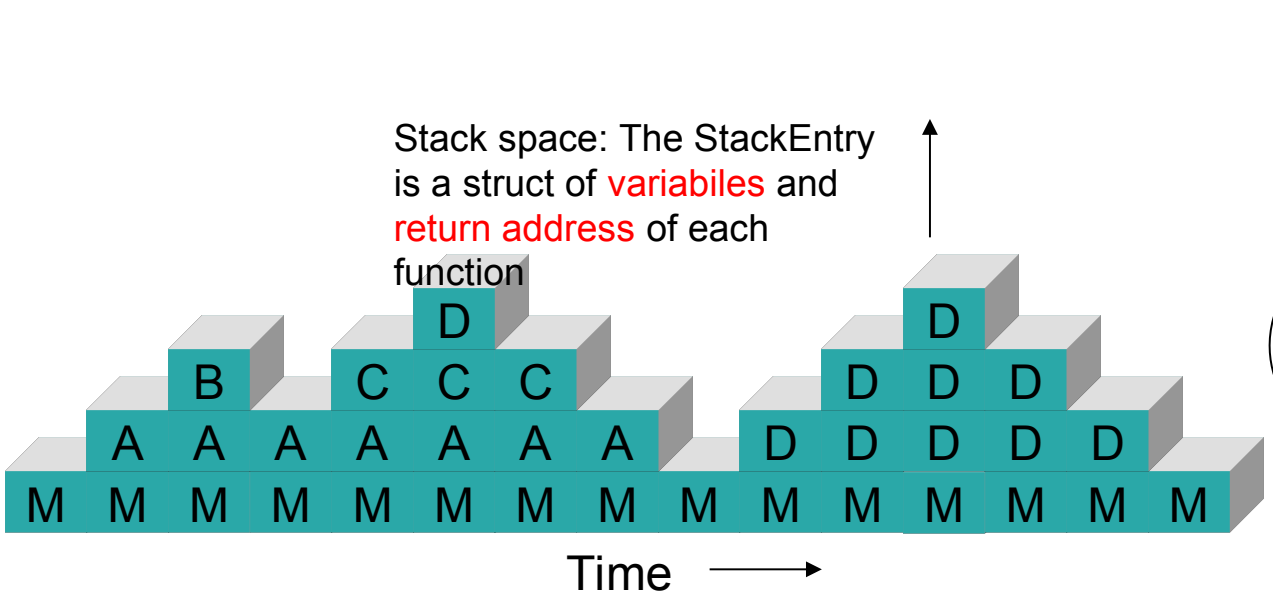
Kruse and Leung "Data Structures & Program Design in C"

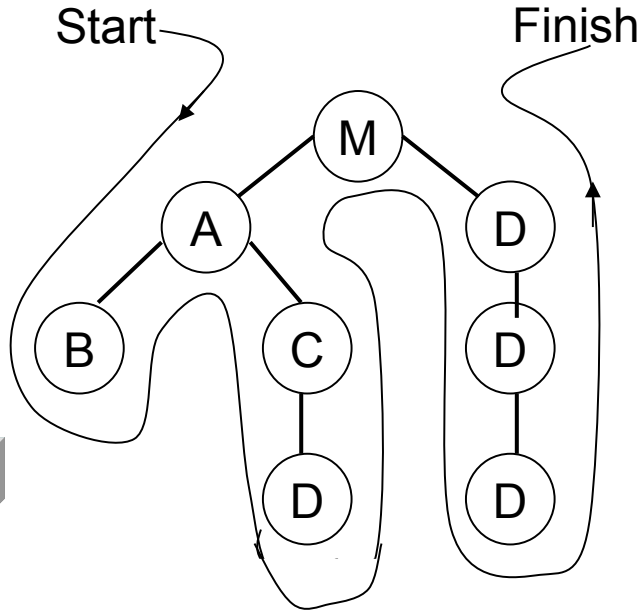
Prepared by:

Waleed A. Yousef, Ph.D.

© Waleed A. Yousef 2008

The great importance of Stacks in OS



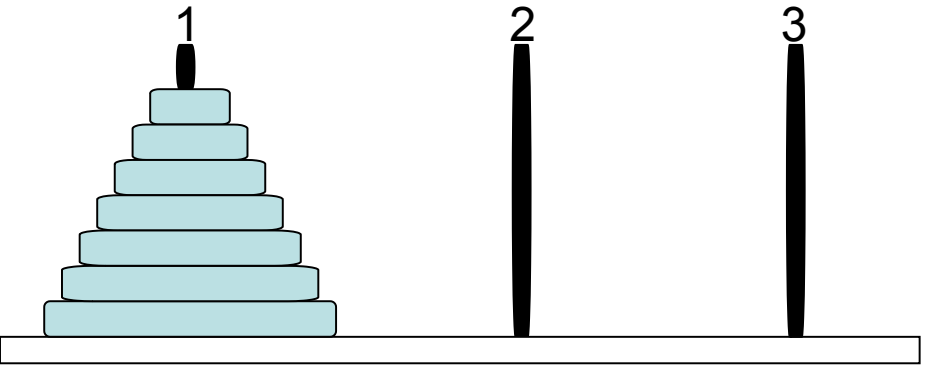


- Program M starts, then calls A, which calls B, which ends then A calls C…
- All local parameters and return address of each function has to be stored.
- Return is in LIFO order.
- Space required is the height of this tree.
- OS has to use a stack to keep track of this storage sequence.

- When a function calls itself, this is recursion.
- Still stack is needed, no difference between A call B, or B calls itself.
- When does a function call itself recursively, or why recursion? See next.

© Waleed A. Yousef 2008

Applications: Towers of Hanoi, a game of 19th century



Write a computer function that writes down the steps needed to move the disks from tower 1 to tower 3 using tower 2 with the restrictions that:

1- Only one disk should be removed at a time.

2- No disk should be placed on top of a smaller disk.

The call of this function will be, e.g., for this figure, `MoveDisks(7, 1, 3, 2)`.

Solution:

1- To move the largest disk from 1 to 3, we have to move first all others from 1 to 2 !!!

2- Move the largest from 1 to 3.

3- Move all others from 2 to 3.

Then the solution is:

```
MoveDisks(6, 1, 2, 3);
"Move disk 7 from 1 to 3"
MoveDisks(6, 2, 3, 1);
```

This is the idea of recursion; define a base step then do the same thing again.

This is the same concept of **divide and conquer**: To solve a problem, we split it into smaller and smaller parts.

Now, how do we code the function `Move`?

© Waleed A. Yousef 2008

Every recursive process consists of two parts:

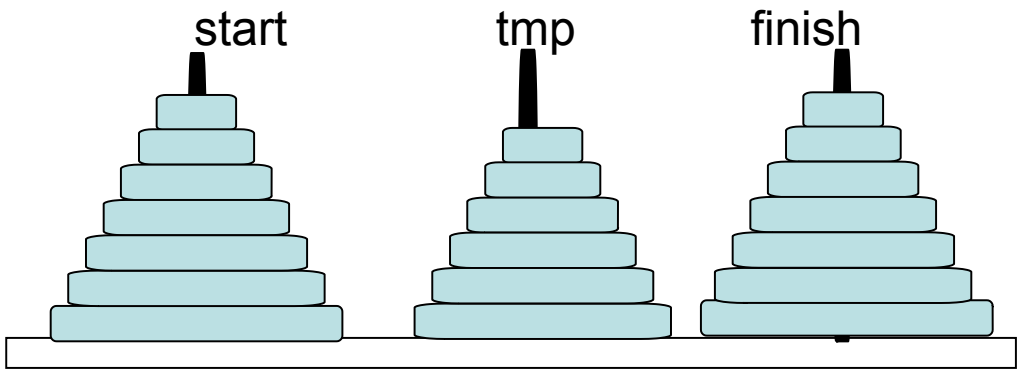
1. A smallest, base case that is processed without recursion.

2. A general method that reduces a particular case to one or more of the smaller cases, thereby making progress toward eventually reducing the problem all the way to the base case.

```
void MoveDisks(int count, int start, int finish, int temp);
/*
Pre:
   - There are at least count disks on the tower start.
   - The top disk on each of towers temp and finish is
   larger than any of the top count disks on tower start.

Post:
   - The top count disks on start have been moved to finish;
   - temp (used for temporary storage) has been returned to
   its starting position.
*/
```

© Waleed A. Yousef 2008



Convince yourself that `MoveDisks` really writes down the steps of moving `count` from `start` to `finish` using `temp`.

```
void MoveDisks(int count, int start, int finish, int temp){
  if (count>0){
    MoveDisks(count-1, start, temp, finish);
    printf("Move disk %d from %d to %d\n", count, start, finish);
    //This step of printf is the base condition.
    MoveDisks(count-1, temp, finish, start);
  }
}
```

© Waleed A. Yousef 2008

MoveDisks (2, 1, 3, 2);

MoveDisks (1, 1, 2, 3)

MoveDisks (0, 1, 3, 2)

"Move disk 1 from 1 to 2."

MoveDisks (0, 3, 2, 1)

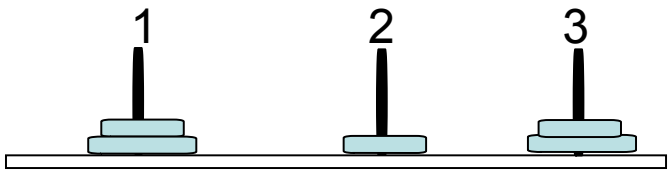
"Move disk 2 from 1 to 3."

MoveDisks (1, 2, 3, 1)

MoveDisks (0, 2, 1, 3)

"Move disk 1 from 2 to 3."

MoveDisks (0, 3, 1, 2)




```
void MoveDisks(int count, int
   start, int finish, int temp){

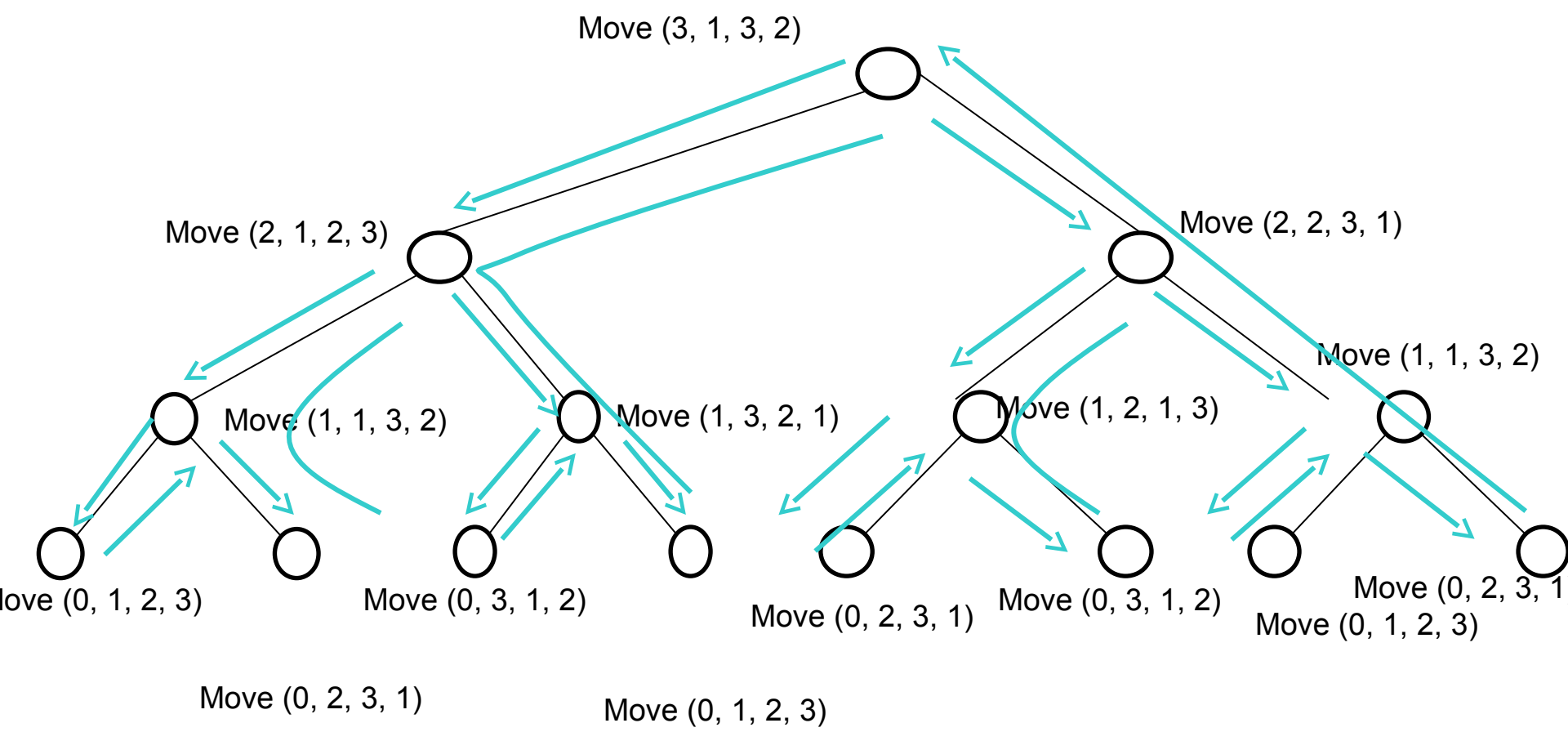
if (count>0){
  MoveDisks(count-1, start, temp,
   finish);

  printf("Move disk %d from %d to
   %d\n", count, start, finish);

   MoveDisks(count-1, temp,
   finish, start);
  }
}
```

Every call to MoveDisks creates a new set of formal parameters with pushing the upper level to the stack.

© Waleed A. Yousef 2008



The largest size of the stack until the function returns to the main program is 4. But the number of function calls is:

$$1 + 2 + 4 + 8 = 1 + 2^1 + \ldots + 2^{count-1} = 2^{count} - 1$$

If count = 64 (for example), there will be no stack overflow problem. But the number of steps = the number of function calls (since every function has one printf) ≅ 1.6 E19 !!!

© Waleed A. Yousef 2008

Can we write the function Move iteratively, i.e., not recursively?

In this case, we will find that we really need to build a stack. We will find that the nature of the problem necessitates that we first solve the problem at size count-1 then return back to the original problem. To return back we had to save it in a stack!

In such cases defining the problem recursively is much better, since you let the OS build the stack itself which is much more efficient than your stack.

However, there are some cases in which solving the problem iteratively is better if the iterative algorithm does not need a stack. Hence, solving it recursively builds an unnecessarily stack, which wastes memory and time consumed in function return. Example is the factorial.

$$\text{Iterative definition}: n! = n \times \left(n-1\right) \times \ldots \times 1.$$

$$\text{Recursive definition}: n! = \begin{cases} 1 & if\ n = 0 \left(\text{base condition}\right) \\ n \times \left(n-1\right)! & if\ n > 0 \end{cases}$$

© Waleed A. Yousef 2008

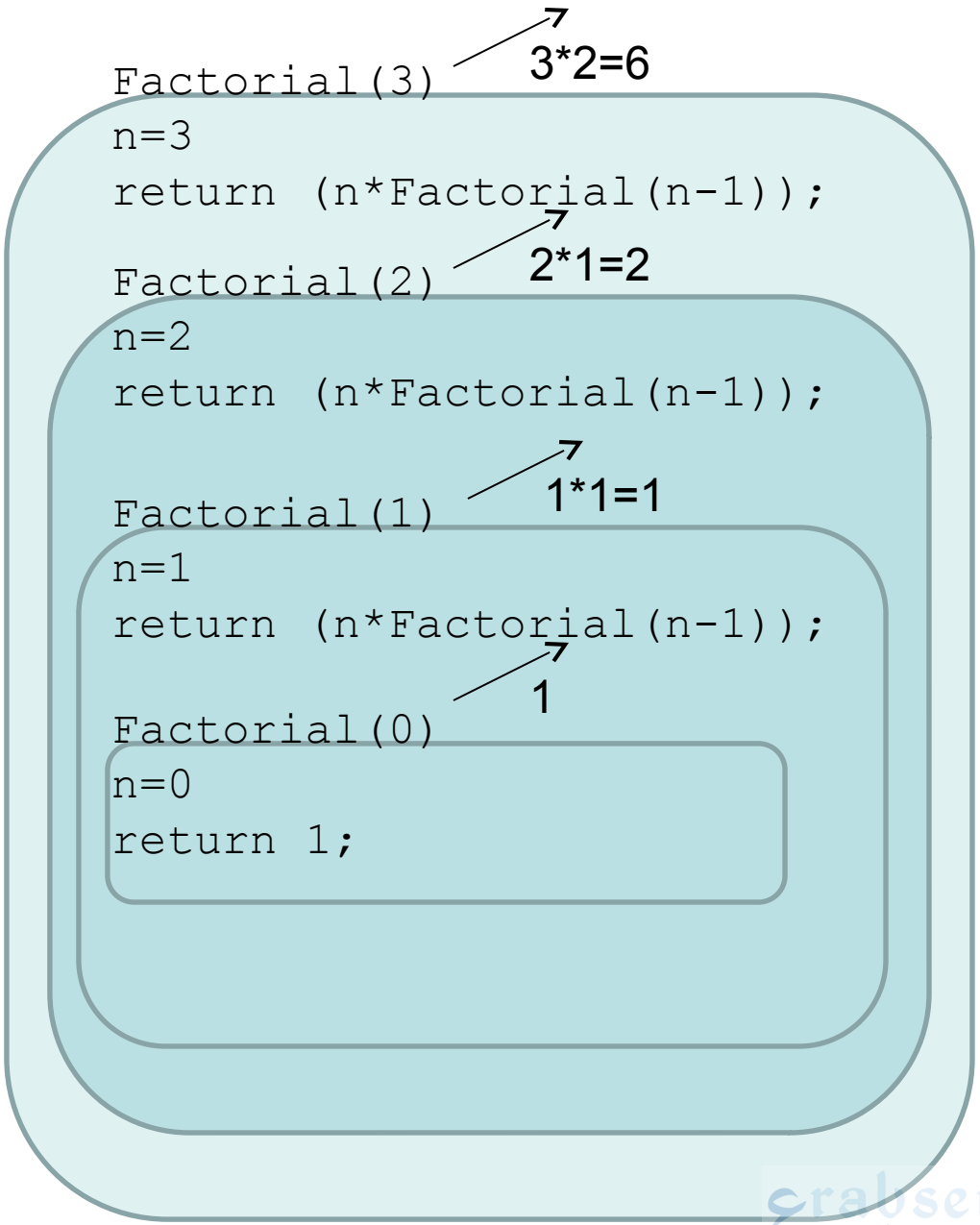
```
/* iterative algorithm*/
int Factorial(int n){
  int count, product;
  for(product=1, count=2; count<=n; count++)
    product*=count;
  return product;
}

/*A smarter version that saves one variable*/
int Factorial(int n){
  int product;
  for(product=1; n>=1; n--)
    product*=n;
  return product;
}
//the decremented n is just a copy; so do not worry
```

The recursive version: First convince yourself that Factorial really returns the factorial except for the base condition:

```
int Factorial(int n){
  return (n==0) ? 1 : (n*Factorial(n-1));
}
```

© Waleed A. Yousef 2008

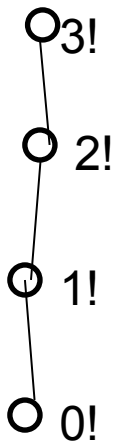



```
int Factorial(int n){
  if(n==0)
    return 1;
  else
    return (n*Factorial(n-1));
}
```

Now:

1- what is the need of this built stack? Why do we exhaust the memory with all these formal parameter?

2- Also, why do we waste the time of function calls and returns. In this case, absolutely the iterative version is faster and more economic in memory utilization.



The recursion tree is simply a chain, which means that designing the algorithm does not need recursion

© Waleed A. Yousef 2008

More extreme example for un-necessary recursion, the Fibonacci function; although many thinks that it is the best example for recursion!!

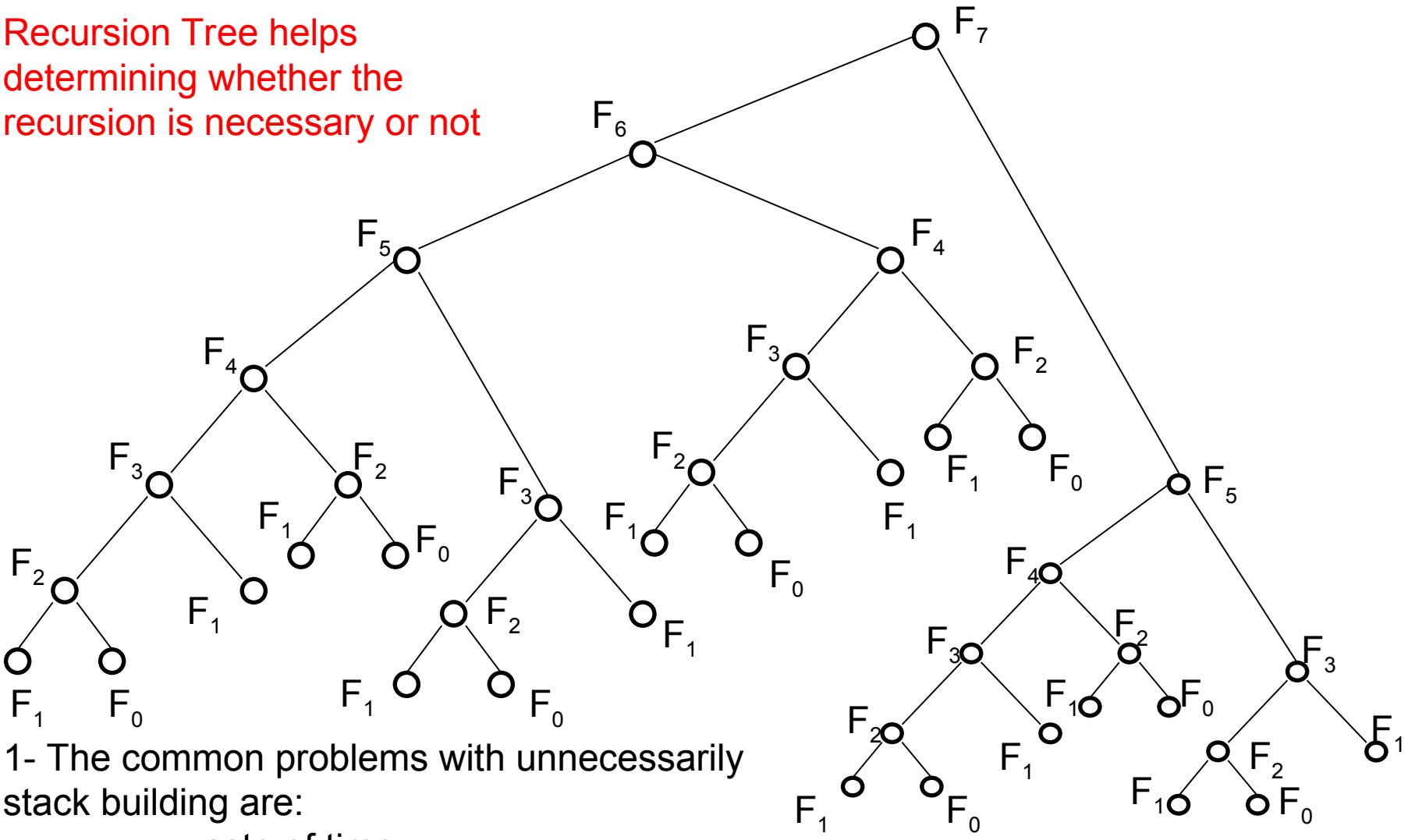
$$F(n) = \begin{cases} 0 & if\ n = 0 (\text{base condition}) \\ 1 & if\ n = 1 (\text{base condition}) \\ F(n-1) + F(n-2) & if\ n \geq 2 \end{cases}$$

Although the mathematical form of the function itself is recursive (this what tempts some people to code it recursively), however we can find an iterative algorithm for solving it. First, we need to analyze the recursive version.

```
int Fibonacci(int n){
  if(n<=0)
    return 0;
  else if (n==1)
    return 1;
  else
    return (Fibonacci(n-1)+Fibonacci(n-2));
}
```

© Waleed A. Yousef 2008

Recursion Tree helps determining whether the recursion is necessary or not



1- The common problems with unnecessarily stack building are:

- waste of time
- waste of memory.

2- For this particular function we have another serious problem, which is needlessly repeating the Fibonacci of many numbers.

The complexity is exponential, i.e., $O(e^n)$

© Waleed A. Yousef 2008

$$F(n) = \begin{cases} 0 & if\ n = 0 (\text{base condition}) \\ 1 & if\ n = 1 (\text{base condition}) \\ F(n-1) + F(n-2) & if\ n \geq 2 \end{cases}$$

0 1 2 3 ....

twoback oneback current

twoback oneback current

```
int Fibonacci(int n){
  int i;
  int twoback;//second previous number, F_i-2
  int oneback;//previous number, F_i-1
  int current;//current number, F_i
  if (n <= 0)
    return 0;
  else if (n == 1)
    return 1;
  else{ //n>=2
    twoback = 0;
    oneback = 1;
    for (i = 2; i <= n; i++) {
      current = twoback + oneback;
      twoback = oneback;
      oneback = current;
    }
    return current;
  }
}
```

The complexity is $O(n)$

© Waleed A. Yousef 2008

Designing Recursive Algorithms

- **Find the key step.** Begin by asking yourself, " How can this problem be divided into parts?" or "How will the key step in the middle be done?"

- **Check termination.** Verify that the recursion will always terminate. Be sure that your algorithm correctly handles extreme cases.

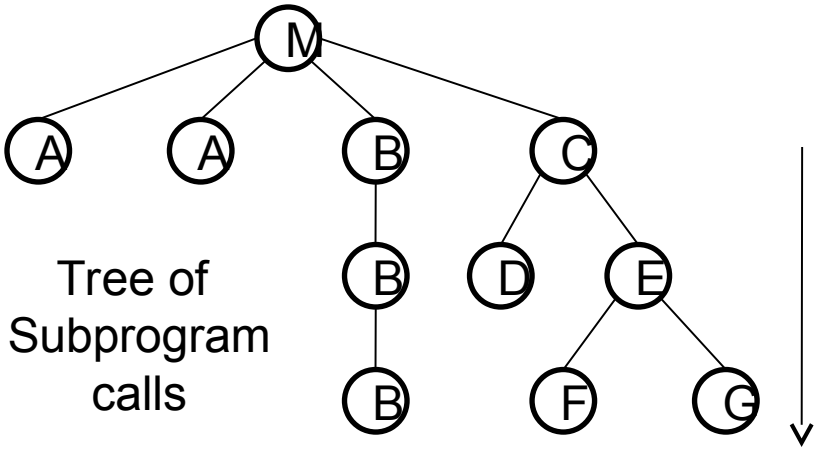
- **Draw a recursion tree.** The height of the tree is closely related to the amount of memory that the program will require, and the total size of the tree reflects the number of times the key step will be done.

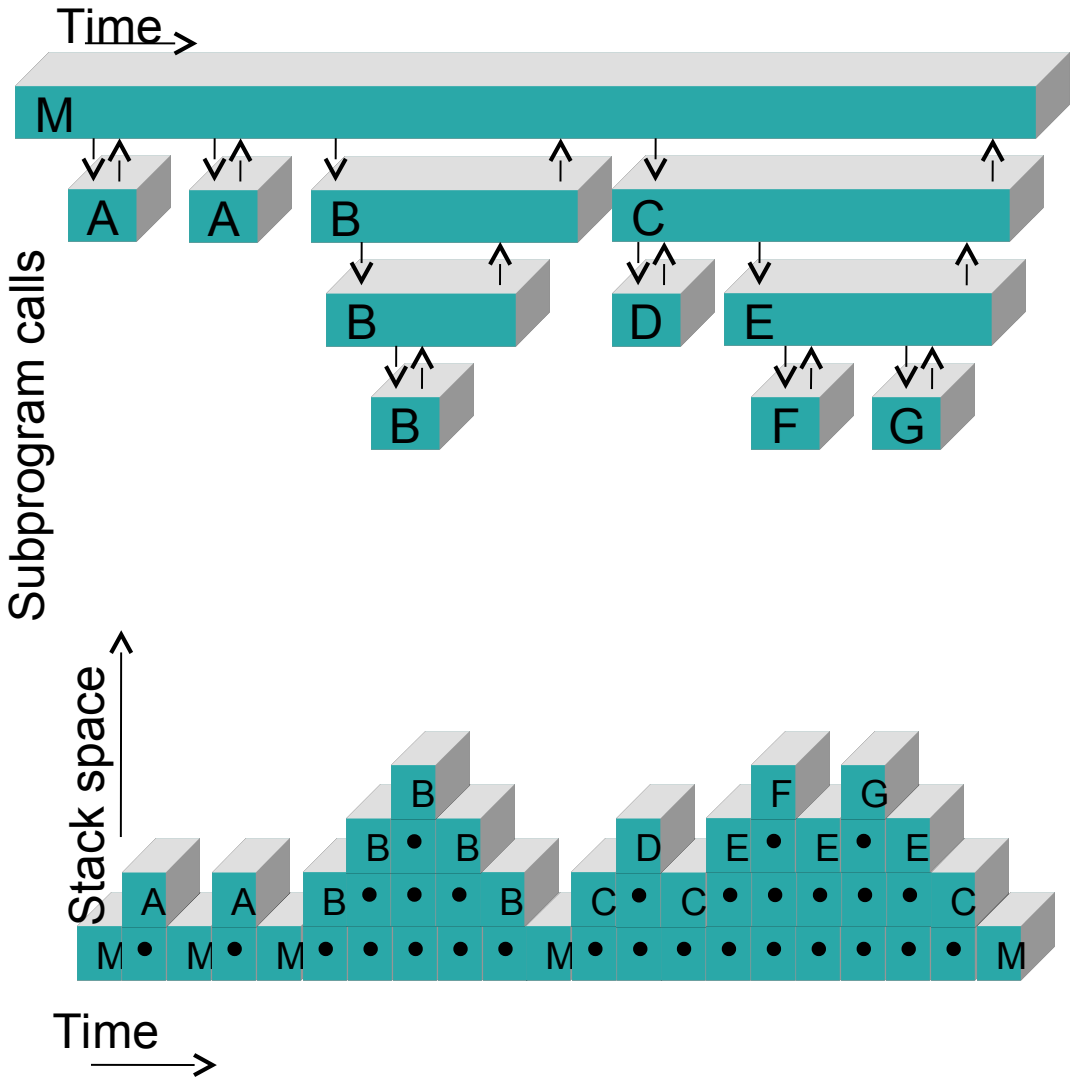
- **Find a stopping rule.** This stopping rule is usually the small, special case that is trivial or easy to handle without recursion.

- **Outline your algorithm.** Combine the stopping rule and the key step, using an if statement to select between them, while convincing yourself that the function performs well in the non-base case

© Waleed A. Yousef 2008

Execution Tree and Stack Size





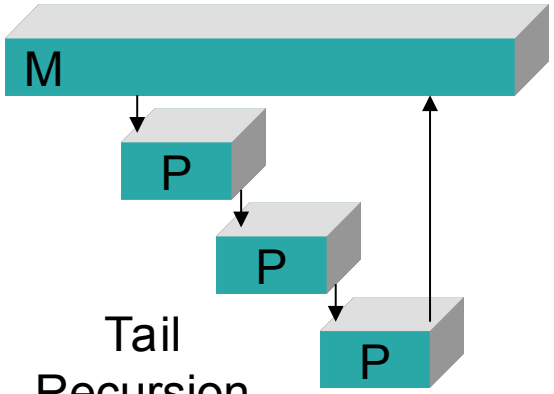
- The OS saves all the parameters of every function and the return address in a stack, so that it returns in LIFO order (See the top of stack in figure).

- The stack size is the depth of the tree

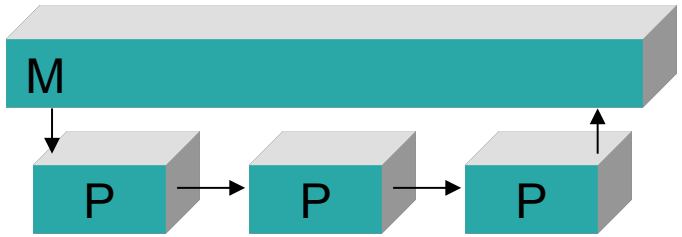
- The number of calls, which determines the complexity and the execution time, is the number of nodes in the tree

© Waleed A. Yousef 2008

Tail Recursion:
Last statement in a function is a call to itself



Tail Recursion



Iterative version of Tail recursion

No need to keep the space of the function in the stack, since it will return to itself. So, just pass the new parameters to the function itself. This reduces the stack size not the complexity of the algorithm

this call can be eliminated by reassigning the calling parameters to the values specified in the recursive call, and then repeating the whole function.

.

© Waleed A. Yousef 2008

With tail recursion:

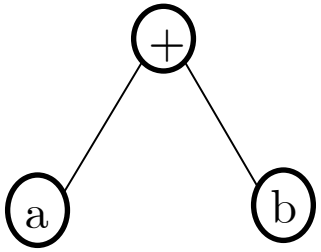
```
void MoveDisks(int count, int start, int finish, int temp){
  if (count>0){
    Move(count-1, start, temp, finish);
    printf("Move disk %d from %d to %d\n", count, start, finish);
    Move(count-1, temp, finish, start);
  }
}
```

Removing the tail recursion:

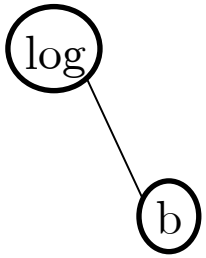
```
void MoveDisks(int count, int start, int finish, int temp){
  int swap;
  while(count>0){// instead of if condition
    MoveDisks(count-1, start, temp, finish);
    printf("Move disk %d from %d to %d\n", count, start, finish);
    count--;
    swap=start;
    start=temp;
    temp=swap;
  }
}
```

© Waleed A. Yousef 2008

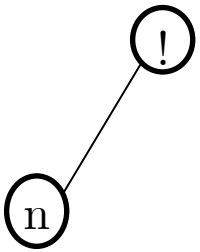
Polish Notation: application on Stacks.



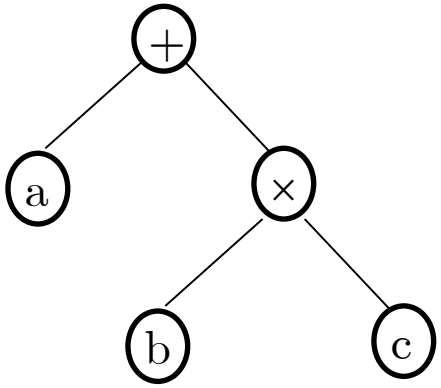
a + b



log b



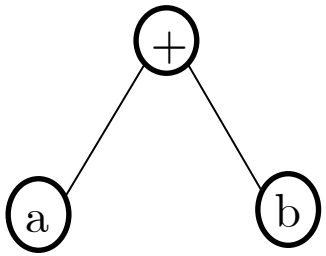
n!



a + b × c

Example:

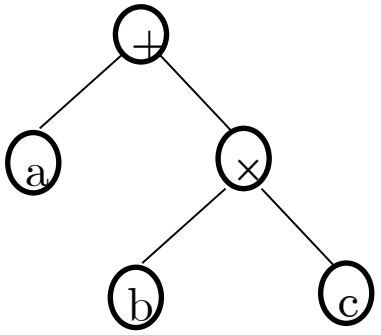
Prefix : +ab
Infix: a+b
Postfix: ab+



a + b

Example:

Prefix : +a×bc
Infix : a+b×c
Postfix : abc×+



a + b × c

We read and write in Infix, however we compute in Postfix! !!

Computer has to convert to, e.g., Postfix before evaluation.

The *Polish Forms* (Ch 12)

© Waleed A. Yousef 2008

How do we really do it?

A + B * C $ D / E – F

A		
A	+	
AB	+	
AB	+*	
ABC	+*	
ABC	+*$	
ABCD	+*$	
ABCD$	+*	/
ABCD$*	+	/
ABCD$*	+/	
ABCD$*E	+/	
ABCD$*E	+/	–
ABCD$*E/	+	–
ABCD$*E/+		–
ABCD$*E/+	–	
ABCD$*E/+F	–	
ABCD$*E/+F–		

More examples:

A+B
AB+

A+B–C
AB+C–

(A+B)*(C–D)
AB+CD–*

A$B*C–D+E/F/(G+H)
AB$C*D– EF/GH+/+

© Waleed A. Yousef 2008

```
void InfixToPostfix(char infix[], char postfix[]){
    int i, j;
    Char op, c;
    Stack s;
    CreateStack(&s);
    for(i=0, j=0; (c=infix[i])!='\0'; i++)
          if(IsDigit(c))
             postfix[j++]=c;
          else{
             if(!StackEmpty(&s)){
                StackTop(&op, &s);
                while(!StackEmpty(&s) && Precedent(op, c)){
                    Pop(&op, &s);
                    postfix[j++]=op;
                    if(!StackEmpty(&s)) StackTop(&op, &s);
                }
             }
             Push(c, &s);
          }
    while(!StackEmpty(&s)){
          Pop(&op, &s);
          postfix[j++]=op;
    }
    postfix[j]='\0';
```

A + B * C $ D / E – F	
A	
A	+
AB	+
AB	+*
ABC	+*
ABC	+*$
ABCD	+*$
ABCD$	+* /
ABCD$*	+ /
ABCD$*	+/
ABCD$*E	+/
ABCD$*E	+/ –
ABCD$*E/	+ –
ABCD$*E/+	–
ABCD$*E/+	–
ABCD$*E/+F	–
ABCD$*E/+F–	

©Waleed A. Yousef 2008

```
int IsDigit(char c){
    return(c>='0' && c<='9');
}
```

```
int Precedent(char op1, char op2){
    if (op1=='$') return 1;

    if ((op1=='*') || (op1=='/'))
        return (op2!='$');

    if ((op1=='+') || (op1=='-'))
        return ( (op2!='$')&&(op2!='*')&&(op2!='/') );

    return 0;
}
```

Obvious Limitations:

1. only single digit integers
2. only positive integers
3. no unary operators allowed.
4. no brackets !!!
5. no handling for invalid `expr`.
   - extra operators.
   - invalid ordering
6. No characters allowed

HW: rewrite it to account for as much as limitations you can! This can be your course project.
Hint: read Polish Notation, Ch.12.

```
void main(){
    char infix[]="1+2*3$4/5+6";
    char postfix[80];
    InfixToPostfix(infix, postfix);
    printf("\n %s", postfix);
    getch();
}
```

© Waleed A. Yousef 2008

```
double EvaluatePostfix(char expr[]){
   int i;
   char c;
   double op1, op2, val;
   Stack s;

   CreateStack(&s);
   for(i=0; (c=expr[i])!='\0'; i++)
      if(IsDigit(c))
         Push((double)(c-'0'), &s);
      else{
          Pop(&op2, &s);
          Pop(&op1, &s);
          val=Oper(c, op1, op2);
          Push(val, &s);
      }
    Pop(&val, &s);
    return val;
}
```

A B C D $ * E / + F –

Obvious Limitations:

1.only single digit integers
2.only positive integers
3.no unary operators allowed.
4.no brackets !!!
5.no handling for invalid `expr`.
   - extra operators.
   - invalid ordering
   …

HW: rewrite it to account for as much as limitations you can! This can be your course project.
Hint: read Polish Notation, Ch.12.

```
void main(){
   char expr[]="354*+2$";
   double x;
   x=EvaluatePostfix(expr);
   printf("\n %f", x);
   getch();
}
```

© Waleed A. Yousef 2008

```
int IsDigit(char c){
   return(c>='0' && c<='9');
}

double Oper(char c, double op1, double op2){
   switch(c){
      case '+': return(op1+op2);
      case '-': return(op1-op2);
      case '*': return(op1*op2);
      case '/': return(op1/op2);
      case '$': return(pow(op1,op2));
   }
}
```

© Waleed A. Yousef 2008

Big Surprise!

We cannot combine both routines, `InfixToPostfix` and `EvaluatePostfix`, in a single program because of different element types!!!

Solutions will be provided in: ("Abstraction and Implementation Related Issues")

Until we reach it, a good programming style to cope with different types and configurations is via compiler directives, e.g., `#define`, `#ifdef`, …

This way, you will change only a single line of code instead changing the whole `Stack.h`

```
                /*Stack.h*/
//#define INFIX_TO_POSTFIX
#define EVALUATE_POSTFIX

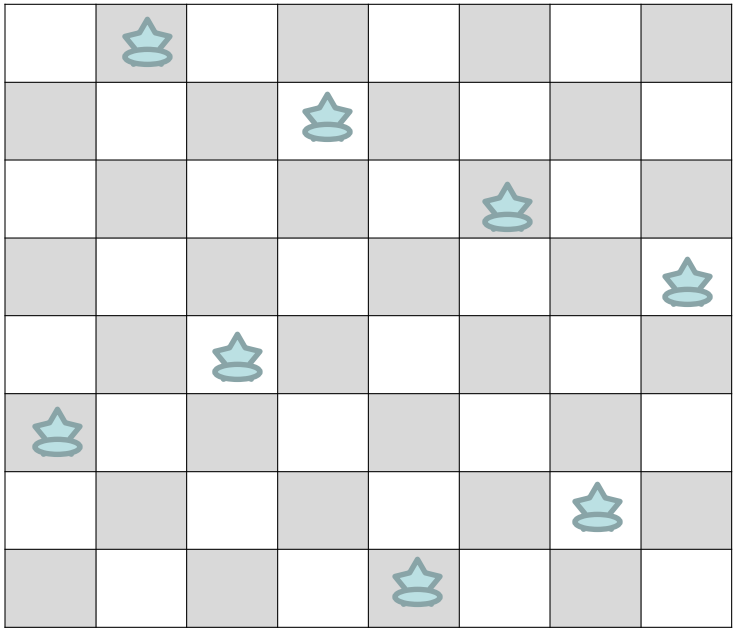
#ifdef INFIX_TO_POSTFIX
      typedef char ElementType;
      #define MAXELEMENTS 100
      typedef ElementType StackEntry;
      #define MAXSTACK MAXELEMENTS
#endif

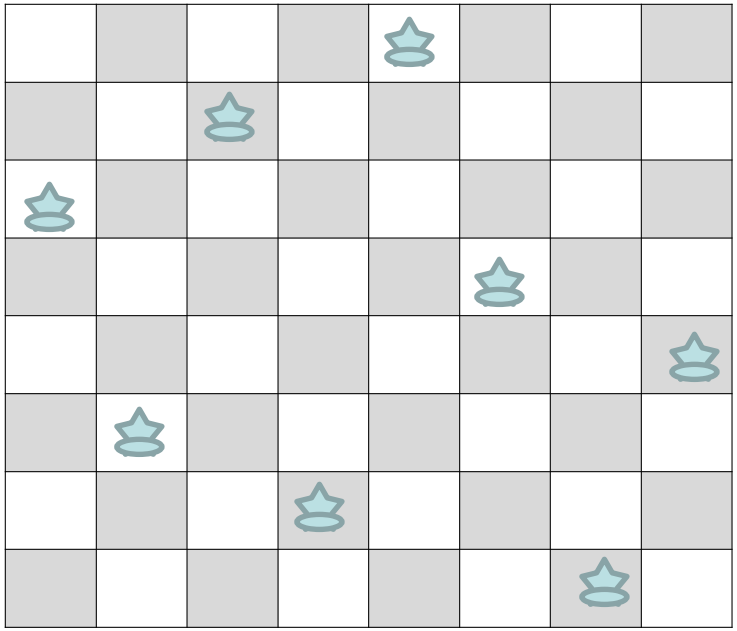
#ifdef EVALUATE_POSTFIX
      typedef double ElementType;
      #define MAXELEMENTS 100
      typedef ElementType StackEntry;
      #define MAXSTACK MAXELEMENTS
#endif
```

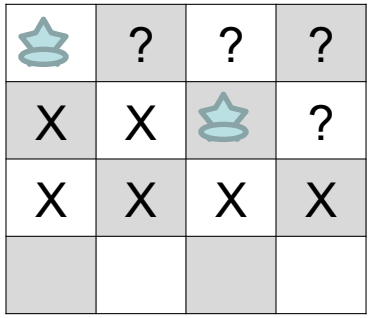
© Waleed A. Yousef 2008

Backtracking (postponing the work): another application for stacks

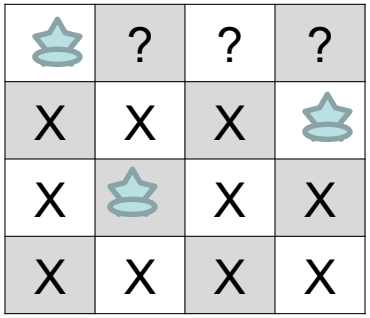
Eight Queens Puzzle (read it yourself)



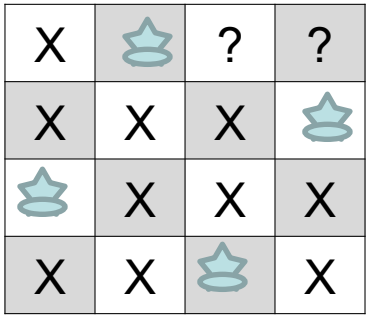




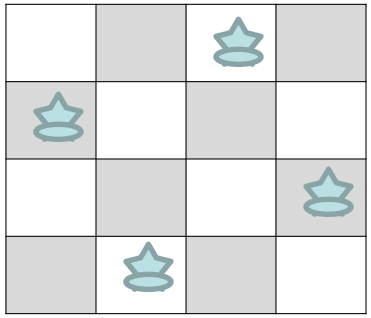
Dead end
(a)



Dead end
(b)



Solution
(c)



Solution
(d)

© Waleed A. Yousef 2008